

MySQL应用教程

第六章 MySQL视图

内容提要

- ▶ 视图的概念
- ▶ 视图的操作
- ▶ 视图的作用

什么叫视图？

- ▶ 视图是一个**虚表**，是从一个或几个基本表（或视图）导出的表
- ▶ **只存放视图的定义**，不存放视图对应的数据
- ▶ 基表中的数据发生变化，从视图中查询出的数据也随之改

视图与基本表的关系

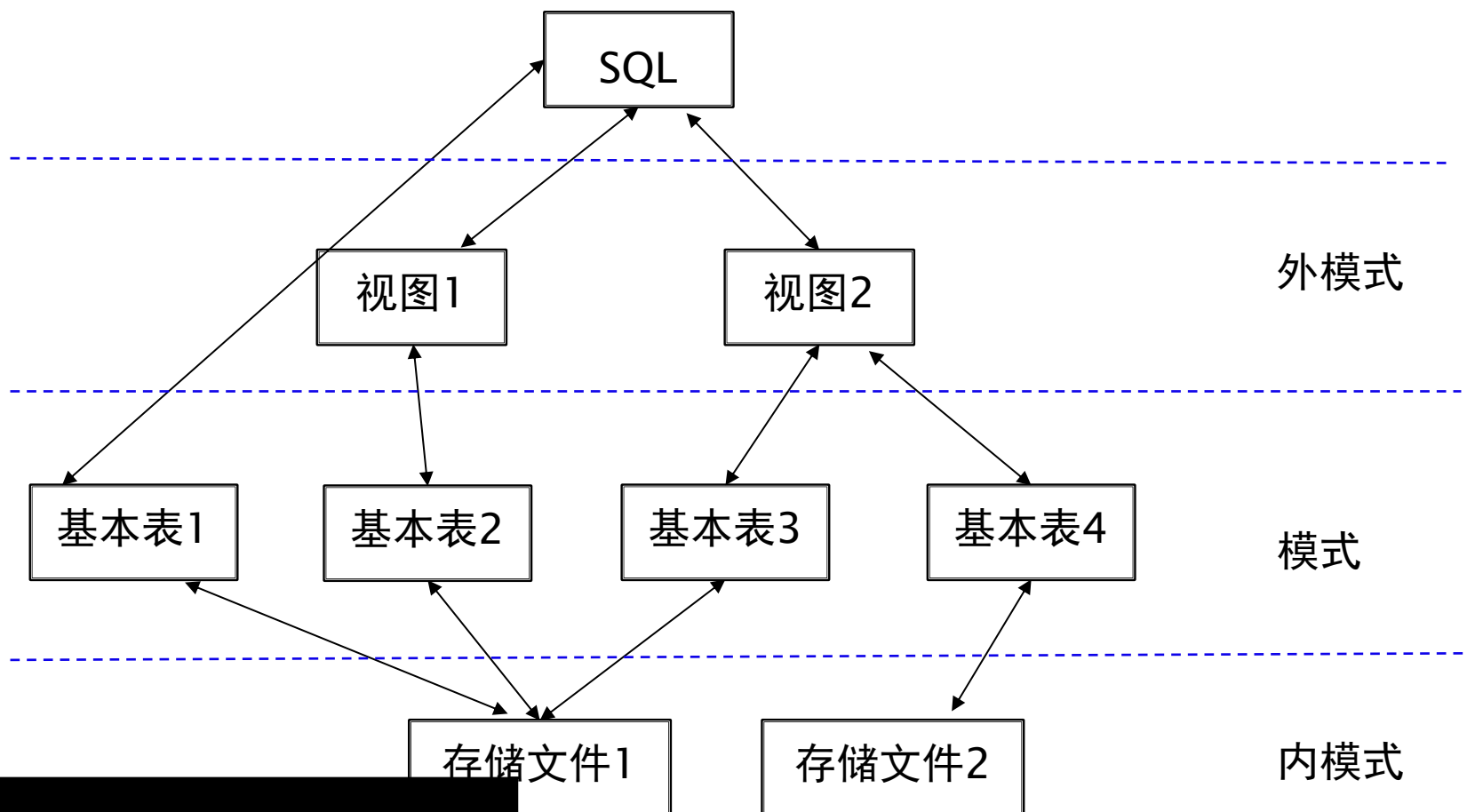
视图

姓名	班级	课程名称	成绩
徐美利	信管 1401	计算机应用软件	85
徐美利	信管 1401	电子商务	75
聂鹏飞	食品 1401	C 语言程序设计	87
聂鹏飞	食品 1401	计算机应用软件	96
聂鹏飞	食品 1401	数据库原理	90

基本表

学号	姓名	班级	课程号	课程名称	成绩
1412855223	徐美利	信管 1401	58130540	计算机应用软件	85
1416855305	聂鹏飞	食品 1401	18110140	C 语言程序设计	87
1114070116	欧阳宝贝	工商 1401	58130060	ASP.NET 程序设计	60
1414855302	李杜	会计 1401	11110470	统计学 A	65
1411855228	唐晓	商务 1401	11110930	电子商务	75
1207040137	张冰霜	信管 1201	18111850	数据库原理	96
...	...	...	...	...	...

SQL语言支持关系数据库三级模式



内容提要

- ▶ 视图的概念
- ▶ **mysql视图的操作**
- ▶ 视图的作用



定义视图

查询视图

修改视图

删除视图

更新视图

1.定义视图

- ▶ 语法格式：

```
CREATE  
VIEW view_name [(column_list)]  
AS select_statement  
[WITH CHECK OPTION]
```

- WITH CHECK OPTION：对视图进行UPDATE, INSERT和DELETE操作时要保证更新、插入或删除的行满足视图定义中的谓词条件；

定义视图（续）

```
CREATE  
VIEW view_name  
[(column_list)]  
AS select_statement  
[WITH CHECK OPTION]
```

■ 组成视图的属性列名：全部省略或全部指定

✓ 全部省略：

由子查询中SELECT目标列中的诸字段组成

✓ 明确指定视图的所有列名：

某个目标列是聚集函数或列表表达式

多表连接时选出了几个同名列作为视图的字段

需要在视图中为某个列启用新的更合适的名字

定义视图（续）

- 关系数据库管理系统执行CREATE VIEW语句时只是把视图定义存入数据字典，并不执行其中SELECT语句。
- 在对视图查询时，按视图的定义从基本表中将数据查出。

定义视图（续）

- ▶ 【例1】 建立信息系学生的视图。

```
CREATE VIEW IS_Student AS  
SELECT SNO,SN,AGE FROM S  
WHERE DEPT='IS';
```

IS_Student视图

```
mysql> desc is_student;
```

Field	Type	Null	Key	Default	Extra
SNO	char(6)	NO		NULL	
SN	varchar(20)	NO		NULL	
AGE	tinyint unsigned	NO		NULL	

S基本表

Field	Type	Null	Key	Default	Extra
SNO	char(6)	NO	PRI	NULL	
SN	varchar(20)	NO		NULL	
AGE	tinyint unsigned	NO		NULL	
DEPT	varchar(20)	YES		NULL	

```
mysql> select * from is_student;
```

SNO	SN	AGE
S2	刘蓝	23
S4	张立伟	19
S5	王世明	19

定义视图（续）

- ▶ 对于【例1】若建立信息系学生的视图，并要求进行修改和插入操作时仍需保证该视图只有信息系的学生。

```
CREATE VIEW IS_Student1 AS  
SELECT SNO,SN,AGE FROM S  
WHERE DEPT='IS'  
WITH CHECK OPTION;
```

定义IS_Student1视图时加上了WITH CHECK OPTION子句，对该视图进行插入、修改和删除操作时，RDBMS会自动加上dept='IS'的条件。

若一个视图是从单个基本表导出的，并且只是去掉了基本表的某些行和某些列，但保留了主码，我们称这类视图为**行列子集视图**。

IS_Student视图就是一个行列子集视图

定义视图（续）

- ▶ 【例2】建立信息系选修了C1号课程的学生视图（包括 学号、姓名、成绩）。

```
CREATE VIEW IS_Student2(SNO,SN,SCORE)AS  
SELECT S.SNO,S.SN,SC.SCORE FROM S  
JOIN SC  
ON SC.SNO=S.SNO  
WHERE S.DEPT='IS' AND CNO='C1';
```



基于多表的视图

定义视图（续）

- ▶ 【例3】 建立信息系选修了c1号课程且成绩在90分以上的学生的视图。

```
CREATE VIEW IS_Student3 AS  
SELECT SNO,SN,SCORE FROM IS_Student2  
WHERE SCORE>=90;
```



建立在视图上的视图

定义视图（续）

- ▶ 【例4】 定义一个反映学生出生年份的视图。

```
CREATE VIEW BT_Student(SNO,SN,BIRTH) AS  
SELECT SNO,SN,2020-AGE FROM S;
```



带表达式的视图

定义视图（续）

- ▶ 【例5】将学生的学号及平均成绩定义为一个视图

```
CREATE VIEW AVG_Student(SNO,AVG) AS  
SELECT SNO,AVG(SCORE) FROM SC  
GROUP BY SNO;
```



分组视图

2. 查询视图

■ 用户角度：

查询视图与查询基本表相同

■ 关系数据库管理系统实现视图查询的方法

视图消解法 (View Resolution)

1. 进行有效性检查
2. 转换成等价的对基本表的查询
3. 执行修正后的查询

查询视图(续)

```
CREATE VIEW IS_Student AS  
SELECT SNO,SN,AGE FROM S  
WHERE DEPT='IS';
```

- ▶ 【例6】在信息系学生的视图中找出年龄小于20岁的学生。

```
SELECT SNO,AGE FROM IS_Student WHERE AGE<20;
```

- ▶ 视图消解转换后的查询语句为：

```
SELECT SNO,AGE FROM S WHERE DEPT='IS' AND AGE<20;
```

3 修改视图

语法格式：

1) **CREATE OR REPLACE** 语句格式：

CREATE OR REPLACE VIEW 视图名 [(视图列表)]
AS 查询语句
[WITH CHECK OPTION];

2) **ALTER**语句, 语法格式：

ALTER VIEW 视图名 [(视图列表)]
AS 查询语句
[WITH CHECK OPTION];



```
CREATE VIEW student_view1(SNAME,CNAME,SCORE) AS  
SELECT SN,CN,SCORE FROM S  
INNER JOIN SC  
ON S.SNO=SC.SNO  
INNER JOIN C  
ON SC.CNO=C.CNO;
```

```
CREATE OR REPLACE VIEW student_view1(姓名,课程名,成绩)  
AS SELECT SN,CN,SCORE FROM S  
INNER JOIN SC  
ON S.SNO=SC.SNO  
INNER JOIN C  
ON SC.CNO=C.CNO;
```

```
ALTER VIEW student_view1(SNAME,CNAME,SCORE)  
AS SELECT SN,CN,SCORE FROM S  
INNER JOIN SC  
ON S.SNO=SC.SNO  
INNER JOIN C  
ON SC.CNO=C.CNO;
```

4. 删除视图

- 删除视图时，只能删除视图的定义，不会删除数据。其次用户必须拥有drop权限。
- 语法格式：**DROP VIEW** [IF EXISTS]
视图名[**CASCADE**];
- 如果该视图上还导出了其他视图，使用CASCADE级联删除语句，把该视图和由它导出的所有视图一起删除

```
DROP VIEW IF EXISTS student_view1;
```

5.视图的更新

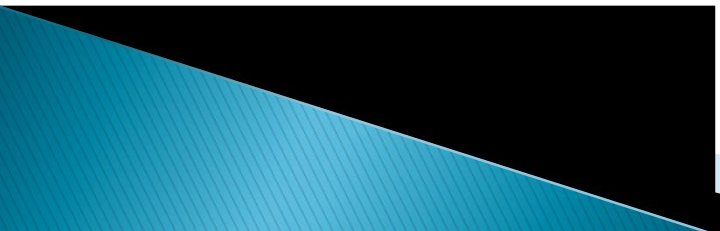
- ▶ 对视图的更新其实就是对表的更新，更新视图是指通过视图来插入（insert）、更新（update）和删除（delete）表中的数据。
- ▶ 通过视图更新时，都是转换到基本表来更新。
- ▶ 更新视图时，只能更新权限范围内的数据。

[例] 将信息系学生视图IS_Student中学号” 201215122”的学生姓名改为” 刘辰。

```
UPDATE IS_Student SET Sname= '刘辰'  
WHERE Sno= ' 201215122 ';
```

转换后的语句：

```
UPDATE Student SET Sname= '刘辰' WHERE  
Sno= ' 201215122 ' AND Sdept= 'IS';
```



[例] 向信息系学生视图IS_Student中插入一个新的学 生记录，其中学号为” 201215129”，姓名为” 赵新”， 年龄 为20岁

```
INSERT INTO IS_Student  
VALUES('201215129','赵新' ,20);
```

转换为对基本表的更新：

```
INSERT INTO  
Student(Sno,Sname,Sage,Sdept)  
VALUES('200215129 ','赵新',20,'IS' );
```

[例] 删除信息系学生视图IS_Student中学号为” 201215129”的记录

```
DELETE FROM IS_Student WHERE Sno= '201215129 ';
```

转换为对基本表的更新：

```
DELETE FROM Student WHERE Sno= '201215129 ' AND Sdept= 'IS';
```



更新视图的限制：一些视图是不可更新的，因为对这些视图的更新不能唯一地有意义地转换成对相应基本表的更新

例：视图AVG_Student为不可更新视图。

```
UPDATE AVG_Student SET AVG=90 WHERE  
SNO= '201215121';
```

这个对视图的更新无法转换成对基本表SC的更新

```
CREATE VIEW AVG_Student(SNO,AVG) AS  
SELECT SNO,AVG(SCORE) FROM SC  
GROUP BY SNO;
```

视图的更新（续）

- 允许对行列子集视图进行更新
- 对其他类型视图的更新不同系统有不同限制

视图的更新（续）

以下情况视图无法更新(尽量不要用视图更新数据)

- ◆ 视图中包含sum(), count()等聚集函数的;
- ◆ 视图中包含union、union all、distinct、group by、having等关键字的;
- ◆ 常量视图, 比如: create view view_now as select now();
- ◆ 由不可更新的视图导出的视图;
- ◆ 视图对应的表上存在没有默认值的列, 而且该列没有包含在视图里;
- ◆ with check option也将决定视图是否可以更新;
- ◆ 视图中包含子查询, 并且内层查询的from子句中涉及的表也是导出该视图的基本表, 则此视图不能允许更新;

内容提要

- ▶ 视图的概念
- ▶ 视图的操作
- ▶ 视图的作用

视图的作用

▶ 增强数据安全性

同一个数据库表可以创建不同的视图，为不同用户分配不同的视图，这样就可以实现不同用户只能查询或修改与之对应的数据，继而增强了数据的安全访问控制。

▶ 适当的利用视图可以更清晰的表达查询

基于多张表连接形成的视图

基于复杂嵌套查询的视图

含导出属性的视图

视图的作用(续)

► 提高数据得逻辑独立性

如果没有视图，应用程序就直接建立在基本表上；有了视图之后，应用程序就可以建立在视图上，从而使应用程序和数据库表结构有一定程度上的逻辑分离。

例：学生关系

Student(Sno,Sname,Ssex,Sage,Sdept)

“垂直”地分成两个基本表：

SX(Sno,Sname,Sage)

SY(Sno,Ssex,Sdept)

通过建立一个视图Student:

CREATE VIEW

Student(Sno,Sname,Ssex,Sage,Sdept) **AS SELECT**
SX.Sno,SX.Sname,SY.Ssex,SX.Sage,SY.Sdept
FROM SX JOIN SY ON SX.Sno=SY.Sno;

使用户的外模式保持不变，用户的应用程序通过视图
仍然能够查找数据